# 1. Полное задание

1

## Объект и результат

Выполнить сопровождение готовой информационной системы своей предметной области. Использовать отдельный учебный стенд варианта: API журнала записей, исходные данные, действующий контракт и исходную коллекцию. Описать объект эксплуатации и организацию поддержки; воспроизвести граничный инцидент; подтвердить причину; выполнить одно исправление и повторные проверки; проверить восстановление данных и оценить качество по фактическим событиям. Готовый API уже предоставлен. Разработка новой ИС и повторная реализация исходного API не требуются.

2

## Индивидуальные условия

Сохранить номер и название области из отдельного списка. При неопределённом названии процесса, обслуживания или ресурса конкретизировать его в паспорте. Использовать только синтетические данные. Стенд моделирует журнал событий этой области, а не все функции реального предприятия. Указать организацию, пользователей, режим работы, последствия отказов и границы модели. Реальная организация допустима при наличии разрешённых обезличенных сведений; иначе явно указать «учебная модель».

3

### Среда и исходные файлы

Windows, .NET SDK 10, Python 3, PowerShell, браузер, Блокнот или редактор C#, Word и PowerPoint. Использовать свой vNN.zip, data.json, context.json, Program.cs, contract.md и baseline.cases.json; отдельно скачать тестовый проект. Проверить версии командами dotnet --version и py -3 --version. При отсутствии установить SDK с https://dotnet.microsoft.com/download/dotnet/10.0 и Python с https://www.python.org/downloads/; в установщике Python включить Python Launcher, завершить установку и открыть новую консоль. Для установки нужны права, предоставленные владельцем компьютера.

ПП ПМ.06 · Методические рекомендации

# 1. Задание — обязательные результаты

1

### Объект и организация сопровождения

Подготовить паспорт системы: назначение, функции, данные, пользователи, инфраструктура, режим работы, критичность и факторы сопровождения; схему взаимодействия со средой. Определить пять ролей поддержки, матрицу RACI, основные, вспомогательные и управляющие процессы с входом, выходом и ответственным; каналы обращений и единую точку регистрации; SLA по критичности и схему взаимосвязи процессов. Число процессов должно совпадать с фактическим перечнем.

2

## Инциденты и изменения

Различить инцидент и сервисный запрос; подготовить таблицу типовых инцидентов, схему жизненного цикла и разобрать три ситуации: потеря граничной записи, недоступность данных и неверный входной параметр. Первые две воспроизвести; третью проверить как корректно обработанный запрос, а не неисправность сервиса. Отделить симптом, гипотезу и подтверждённую причину. Оформить запрос изменения: цель, область, риск, согласование, тесты, откат и приёмка; схему управления изменениями. Обновить только затронутые документы.

3

## Контракт и контрпример

GET /api/records без параметра возвращает все записи по id. Необязательный end — одна дата YYYY-MM-DD: включить все записи с date меньше либо равной end. Ответ — массив id, title, date. Нет совпадений — 200 и []. Ошибочный, пустой, повторный end или неизвестный параметр — 400 JSON. Недоступный/невалидный файл — 500 и error=data_unavailable. /health даёт 200 при читаемых данных и 503 при отказе. Добавить в коллекцию точный граничный случай: до исправления он не проходит, после проходит. Действующий контракт сохранить. Это учебное эксплуатационное условие, не дословное задание КИМ.

# 1. Задание — непрерывность и сдача

1

## Данные, качество и собственное изменение

До правок сохранить исходный код и данные отдельно. Проверить отказ данных, готовность, восстановление из копии и повтор запросов. Описать регламент резервирования и восстановления, журналы, мониторинг и пороги. Рассчитать реакцию, восстановление, долю успешных проверок и наблюдаемую доступность по объявленным событиям и единицам. Для SLA использовать пределы своей карточки, не выдумывать фактическое время. Перенести третью запись на конечную дату, заранее обновить ожидание коллекции и подтвердить три записи вместо двух. Затем вернуть исходные данные и выполнить чистый повтор.

2

## Доказательства

Сохранить исходный и исправленный Program.cs, исходные данные и резервную копию, cases.json, вывод тестов до/после и при изменённом условии, events.jsonl, заполненные паспорт, RACI, процессы, SLA, три карточки ситуаций, запрос изменения, схемы и инструкцию пользователю. Для выводов различать наблюдения, рассчитанные показатели и предложения. Результаты модели не приписывать реальному предприятию.

3

### Сдать комплект

Передать исходный проект и README с запуском/остановкой/проверкой/откатом, коллекцию и журналы, Отчет о выполнении.docx с рисунками и ссылкой на доступный репозиторий, индивидуальное задание, аттестационный лист, договор, справку, пояснительную записку и презентацию из карточки. Презентацию заполнить собственными результатами и сохранить также PDF. Подписи и реквизиты организации заполняются по фактическим сведениям. Учебный образец ниже не является второй обязательной работой.

ПП ПМ.06 · Методические рекомендации

## 2. Файлы для работы

1

### Список предметных областей и отдельные стенды

Список предметных областей и отдельные стенды

2

### Тестовый проект — HTTP-коллекция и запуск проверок

Тестовый проект — HTTP-коллекция и запуск проверок

3

### Отчет о выполнении.docx — чистая редактируемая форма

Отчет о выполнении.docx — чистая редактируемая форма

4

### Какой комплект использовать

Из списка скачать только назначенный стенд vNN.zip. Тестовый проект распаковать рядом в папку checks. Готовые официальные формы скачать кнопкой «Скачать практику» в своей карточке https://praktik.mayventra.ru/#practice/887957b5486f428493bd40df36e6f60d/release . Их назначение отличается от общего отчёта. Далее разобран вариант 01 как пример способа; выполнять только свой вариант. Если уже принят собственный API ПМ.06, входной контракт и коллекцию сначала сверить с руководителем; без подтверждённого соответствия использовать предоставленный самостоятельный стенд. Результат другой практики не требуется.

ПП ПМ.06 · Методические рекомендации

# 3. Методика — подготовка и запуск

1

### 3.1. Распаковать свой стенд

Открыть скачанный ZIP в Проводнике → «Извлечь всё» → выбрать отдельную рабочую папку, например C:\Practice06\v01. В папке должны непосредственно находиться Operations.csproj и Program.cs. В адресной строке Проводника набрать powershell и нажать Enter. Выполнить команды ниже. Если проект не найден, выполнить dir и перейти cd в папку с Operations.csproj. Если порт занят, использовать 5187 во всех адресах и командах этой работы; чужой процесс не останавливать.

2

### Первый запуск

```
dotnet --version
py -3 --version
dotnet run --urls http://127.0.0.1:5186
```

3

### 3.2. Открыть пульт

Оставить консоль работающей. В браузере открыть http://127.0.0.1:5186 . Номер и область в верхней строке должны совпасть с назначением. Нажать «Отправить запрос» при пустом поле: HTTP 200 и три записи. Нажать «Проверить готовность»: HTTP 200 и status=ok. Пустое поле пульта отправляет запрос без end; запрос с явно пустым end проверяется отдельно в коллекции. После изменения кода остановить свой сервер Ctrl+C в его консоли и снова выполнить dotnet run; обновить страницу браузера.

4

### Исходный ответ образца: все три записи, API уже работает.

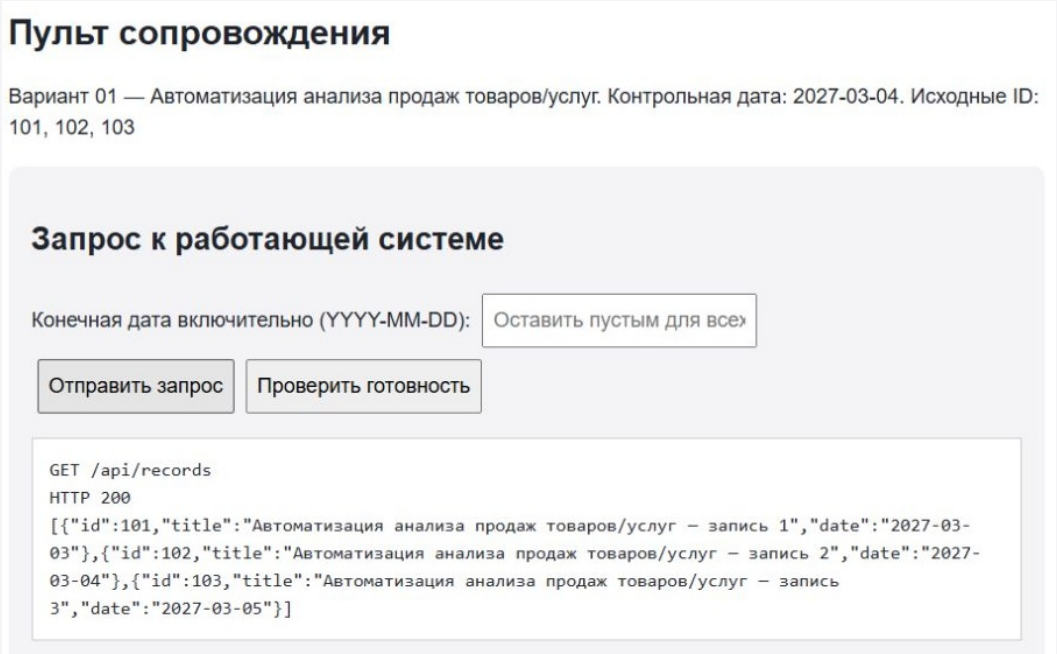



Рисунок 1 — Исходный ответ образца: все три записи, API уже работает.

# 3. Методика — паспорт и границы

1

## 3.3. Заполнить паспорт

Открыть чистую форму отчёта в Word, сохранить отдельную рабочую копию. В исходных условиях указать область, номер, учебную организацию, data.json, contract.md, среду и свой адрес. В этапе «Объект» описать: кто использует журнал, какую операцию обеспечивает каждая запись, какие данные хранятся и что перестанет выполняться при ошибке. Пример: оператор продаж сверяет ежедневные события; пропуск последнего дня искажает список для сверки. Это модель, а не наблюдение на предприятии. Для своей области заменить оператора и назначение записи конкретными ролями.

2

## 3.4. Изобразить среду

В PowerPoint создать отдельный рабочий слайд: Вставка → Фигуры → прямоугольник. Сделать блоки «Пользователь», «Браузер», «API», «data.json», «Журнал событий», соединить стрелками. Подписать пользовательский запрос, чтение данных и запись журнала. Эта контекстная схема не называется BPMN или UML. Для собственного предприятия добавить только реально заявленные внешние компоненты и отметить границу модели. Скопировать схему в предусмотренное место отчёта. Не считать API всей системой предприятия.

3

## Проверка паспорта

Из паспорта понятны объект, владелец данных, потребители, рабочее время и последствия отказа. Для примера принять режим 09:00–18:00, критичность P2: искажён список, общий доступ сохранён. Собственный режим объявить до измерений. Пределы реакции и восстановления взять из своей карточки. Модель предполагает локальные файлы, один оператор и HTTP только на 127.0.0.1; производственное развёртывание здесь не выполняется.

# 3. Методика — служба сопровождения

1

## 3.5. Роли и ответственность

В разделе организации отчёта заполнить пять ролей: пользователь сообщает симптом; первая линия регистрирует обращение; специалист сопровождения проверяет код; администратор отвечает за копии и запуск; руководитель принимает изменение. Один обучающийся может моделировать роли, но не изображает реальные согласования. RACI: R выполняет, A отвечает за итог (один A на строку), C консультирует, I получает информацию. Для регистрации назначить первую линию R/A; для исправления специалиста R, руководителя A; для восстановления администратора R, руководителя A. Остальные роли распределить по участию.

2

## 3.6. Процессы и каналы

Составить таблицу «процесс — класс — вход — выход — ответственный». Сохранить 5 основных: регистрация, обработка, восстановление доступа, обновление, восстановление данных; 5 вспомогательных: резервирование, консультация, тестирование, инструкции, учётные записи; 4 управляющих: SLA, повторные проблемы, план обновлений, доступность. Итого 14 процессов. Для неподдерживаемой функции указать внешнюю границу и ответственного в организации; не придумывать реализацию в учебном API. Все почтовые, телефонные и устные обращения переносить в единый журнал с номером, временем, симптомом, приоритетом и статусом.

3

### 3.7. SLA и схема

Записать рабочее время и правила приоритетов до испытания. Для воспроизводимого P2 использовать reaction_minutes и recovery_minutes своего context.json; это пределы учебной модели в минутах. Для P1 (весь сервис недоступен) принять более строгий предел и обосновать его последствиями; P3 (консультация без отказа) — согласованный срок. Нарисовать блоки «Канал → Регистрация → Классификация → Исполнитель → Проверка пользователем → Закрытие»; от классификации сделать подписанные ветви «Инцидент» и «Сервисный запрос», от инцидента — «Нужно изменение» к согласованию. По схеме должно быть видно, кто получает нерешённое обращение.



# 3. Методика — воспроизвести инцидент

1

### 3.8. Сначала вычислить ожидание

Открыть data.json в Блокноте через Файл → Открыть, ввести полный путь файла. Найти date каждой записи и конечную дату end из context.json. В примере end=2027-03-04: ID 101 за 03.03 включается, 102 за 04.03 включается, 103 за 05.03 исключается. Ожидаются [101,102]. Для своей карточки подставить её дату и первые два ID. Не брать ожидаемый ответ из ошибочного API.

2

### 3.9. Выполнить точный запрос

В поле пульта выделить прежнее значение Ctrl+A и ввести конечную дату. Нажать «Отправить запрос». При исходном коде остаётся только первый ID — это симптом. Нажать «Инцидент обнаружен», затем при начале разбора «Работа принята». Эти события сохраняют фактическое UTC-время в events.jsonl. В карточке инцидента записать ожидание, факт, влияние, приоритет и время. Не нажимать события задним числом.

3

**Граничная запись отсутствует при HTTP 200. Один код успеха не доказывает правильность данных.**

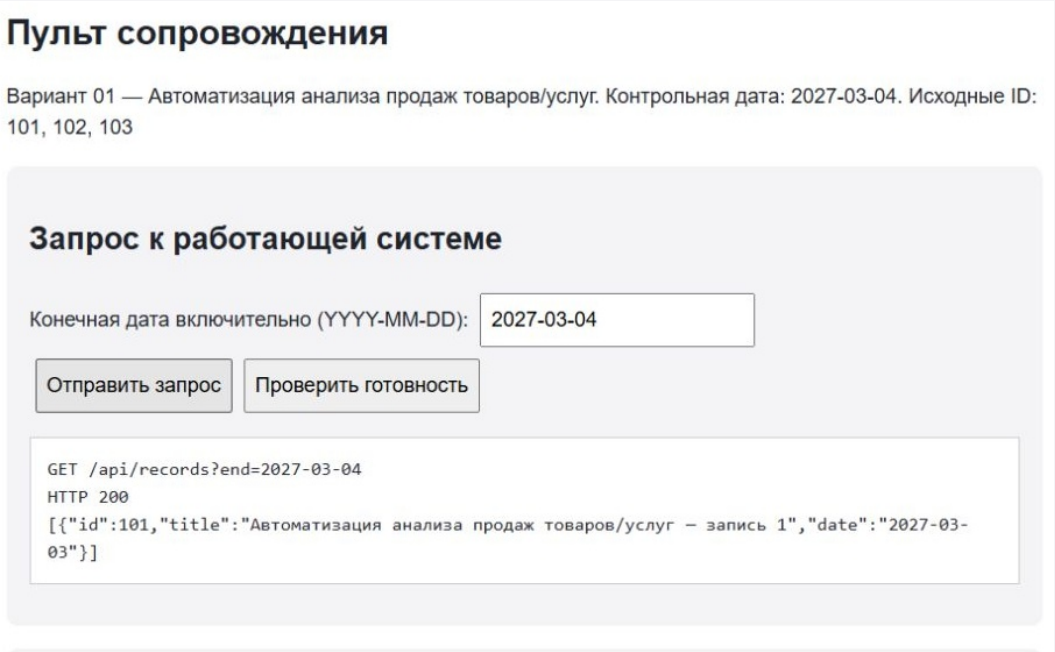



Рисунок 2 — Граничная запись отсутствует при HTTP 200. Один код успеха не доказывает правильность данных.

4

## 3.10. Установить причину

Открыть Program.cs, найти строку var result=all.Where. В ней дата записи сравнивается с end оператором <. Для равной даты выражение ложно, хотя contract.md требует включить день. Журнал запроса подтверждает вход, статус и полученные ID; код подтверждает условие причины. Это дефект предиката отбора, не потеря записи в файле. В карточке отдельно записать первоначальную гипотезу и подтверждение.

# 3. Методика — коллекция до исправления

1

## 3.11. Подготовить тесты

Распаковать отдельный test-project.zip в checks. Скопировать baseline.cases.json своего стенда в checks и переименовать копию в cases.json. Открыть cases.json в Блокноте. Это JSON-массив: элементы разделяются запятыми, последний не имеет запятой. Перед закрывающей ] после последнего объекта добавить запятую и объект ниже. В своём варианте заменить дату и ID. Сохранить UTF-8.

2

## Новый контрпример для варианта 01

```
{"name":"Final date included","path":"/api/records?end=2027-03-04","status":200,"ids"
:[101,102]}
```

3

## 3.12. Запустить отдельный проект

Открыть вторую PowerShell из папки checks. Выполнить команду ниже. Четыре исходных случая проходят, Final date included даёт FAIL и итог FAILED 1 OF 5. Сохранить вывод как before.txt. Если получена ошибка JSON, проверить запятую между объектами; если connection refused — проверить первую консоль и адрес. Тесты сравнивают статус и точные ID; неизменность остальных полей отдельно сверить с data.json.

4

## Команда и сохранение вывода

```
py -3 check.py http://127.0.0.1:5186 cases.json > before.txt
Get-Content before.txt
```

# 3. Методика — изменение и откат

1

## 3.13. Оформить запрос изменения

Записать CR-01: включить конечную дату без изменения пути, параметра и формата. Область: один предикат Program.cs. Риск: включение записей после границы; контроль: точная граница, до всех дат, после всех дат, общий запрос, ошибочная дата. До изменения скопировать Program.cs и data.json в отдельную папку backup; открыть копии и сверить размеры. В учебном запросе указать моделируемое согласование и критерии приёмки, не подделывать подпись. Нажать «Изменение начато». Изобразить процесс «Запрос → Оценка риска → Согласование → Копия → Правка → Тесты → Приёмка» и ветвь «Тест не прошёл → Откат».

2

## 3.14. Внести локальную правку

В Блокноте нажать Ctrl+H. Найти точный фрагмент « < end», заменить на « <= end» только в строке var result. Первая кнопка «Заменить» может только выделить найденный фрагмент; убедиться, что в коде действительно появился знак =. Закрыть панель поиска и сохранить Ctrl+S. Остановить свой сервер Ctrl+C, выполнить прежний dotnet run, обновить пульт. Повторить тот же запрос и коллекцию: два ID и FAILED 0 OF 5. Нажать «Работа восстановлена», затем после проверки «Результат подтверждён».

3

## Условие до и после

```
До: датаЗаписи < end
После: датаЗаписи <= end

В Program.cs датаЗаписи вычисляется выражением:
DateOnly.ParseExact(e.Date,"yyyy-MM-dd",
    CultureInfo.InvariantCulture)
```

4

## Исправлен один оператор: конечная дата включается в результат.

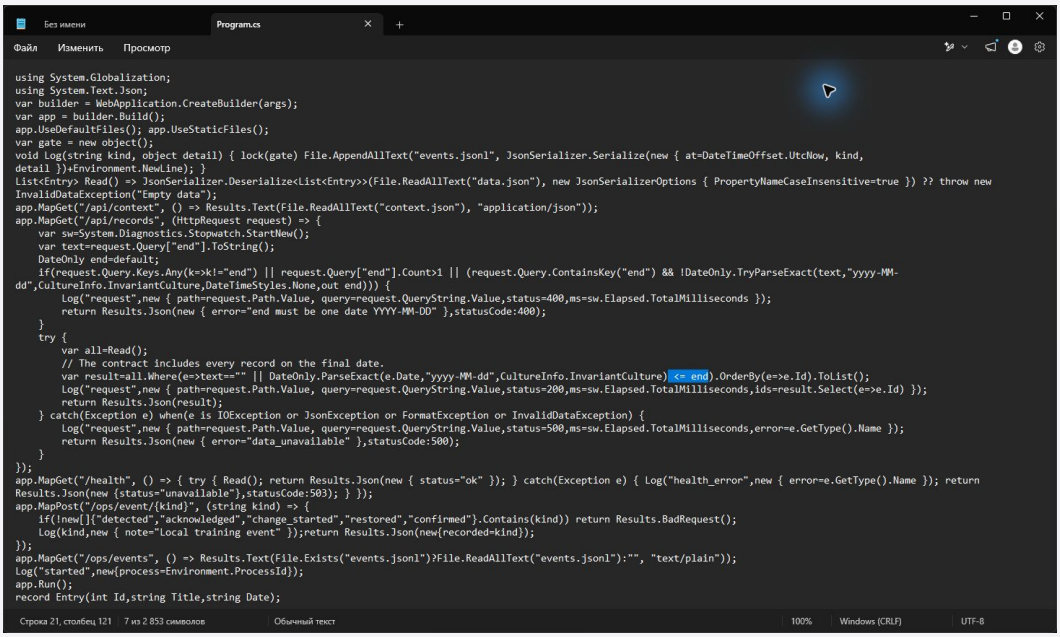



Рисунок 3 — Исправлен один оператор: конечная дата включается в результат.

5

## 3.15. Проверить откат

В отдельной копии рабочего проекта вернуть сохранённый исходный Program.cs, запустить её после остановки своего сервера. Тот же контрпример снова должен дать FAIL. Вернуть исправленный файл, перезапустить, повторить тест до PASS. Откат кода не заменяет восстановление данных. Сохранить версии файла и выводы; исходный скачанный архив оставить без изменений.

# 3. Методика — принять исправление

1

### После перезапуска тот же запрос возвращает оба ожидаемых ID.

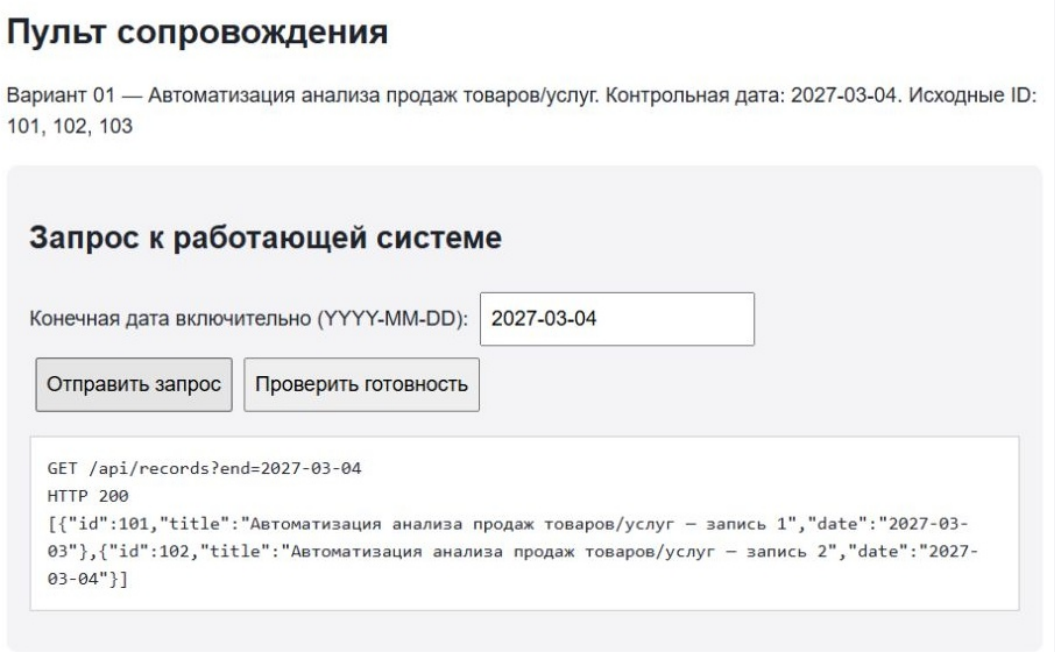



Рисунок 4 — После перезапуска тот же запрос возвращает оба ожидаемых ID.

2

### 3.16. Обновить затронутые документы

В журнале инцидента связать INC-01 и CR-01. В contract.md не менять требование включительной границы: оно уже верно. Добавить в инструкцию пользователю пример конечной даты и пояснение, что HTTP 200 требует проверки состава. В README перечислить новый граничный тест и способ отката. Указать, что исправлено расхождение реализации, а не изменён контракт. Исходную документацию и всё приложение повторно не переписывать.

# 3. Методика — отказ и восстановление

1

## 3.17. Проверить собственную копию

Сохранить data.json в отдельную папку backup. Открыть её и проверить три ID/даты. В рабочей папке в Проводнике выбрать data.json → F2 → Ctrl+A → data.off → Enter; подтвердить изменение расширения. Это контролируемый отказ только учебной копии. Выполнить запрос в пульте: HTTP 500 и data_unavailable; «Проверить готовность» — HTTP 503. Сохранить оба состояния и сообщение журнала, не делать вывод о поломке всей сети.

2

## При недоступном файле API возвращает явную ошибку данных.

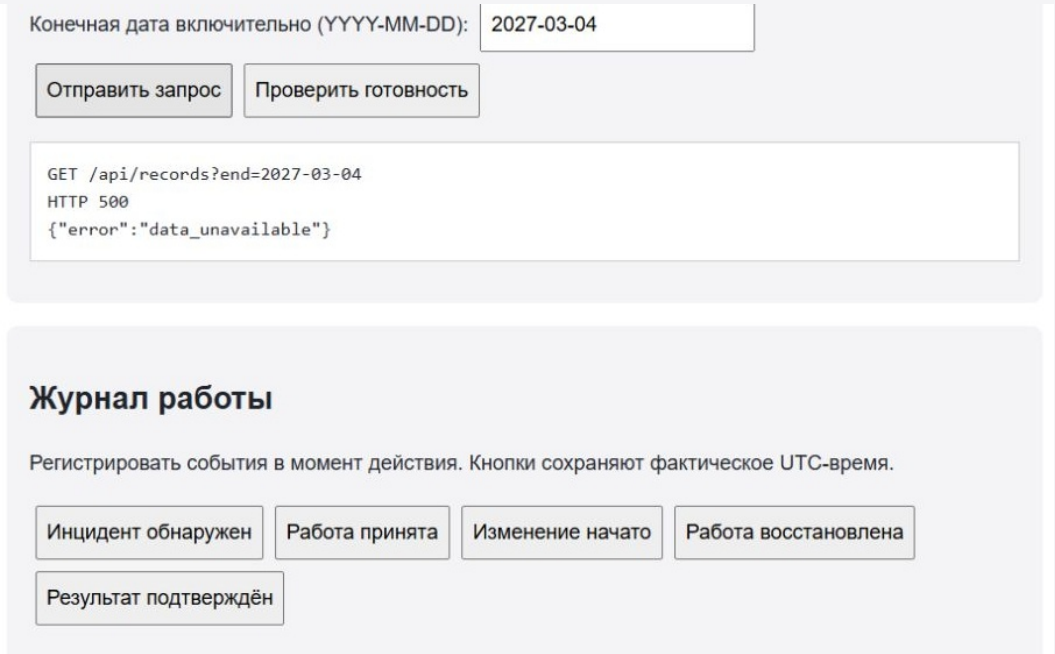



Рисунок 5 — При недоступном файле API возвращает явную ошибку данных.

3

### Проверка готовности различает запуск процесса и доступность данных.

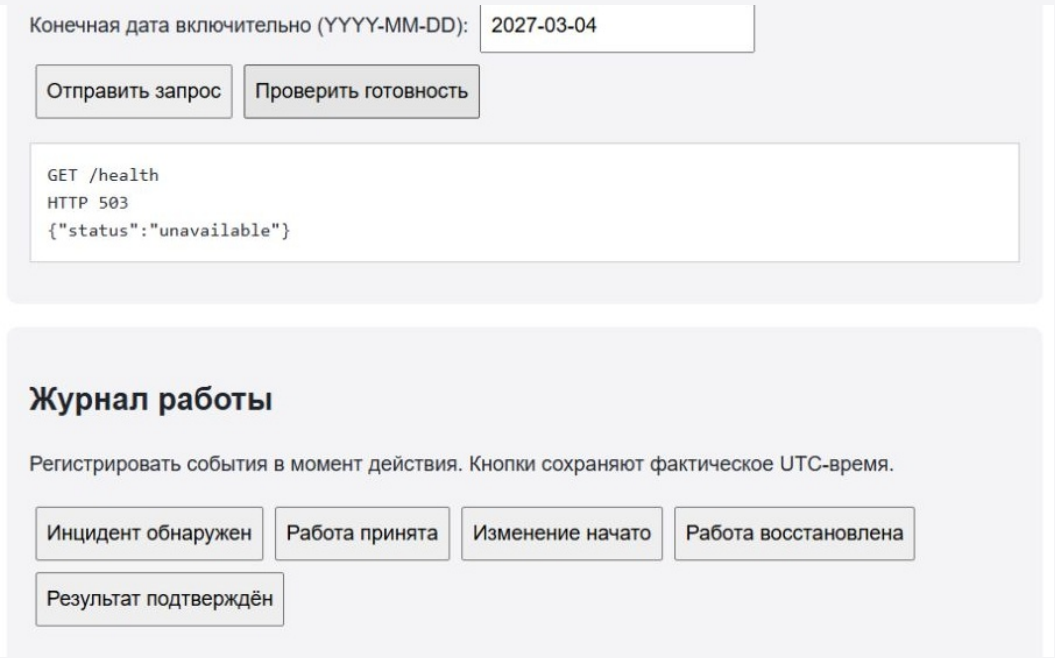



Рисунок 6 — Проверка готовности различает запуск процесса и доступность данных.

ПП ПМ.06 · Методические рекомендации

# 3. Методика — восстановить и измерить

1

### 3.18. Восстановить данные

Из папки backup скопировать проверенный data.json в рабочую папку. data.off оставить как доказательство отказа. Проверить готовность: 200; затем общий запрос: три прежних ID и неизменные title/date; граничный запрос: два ID. Выполнить коллекцию. Сравнить файлы командой Get-FileHash .\data.json и Get-FileHash .\backup\data.json: значения Hash должны совпасть. Резервная копия на том же компьютере моделирует процедуру, но не защищает от потери всего устройства.

2

**Готовность возвращается после восстановления из копии.**

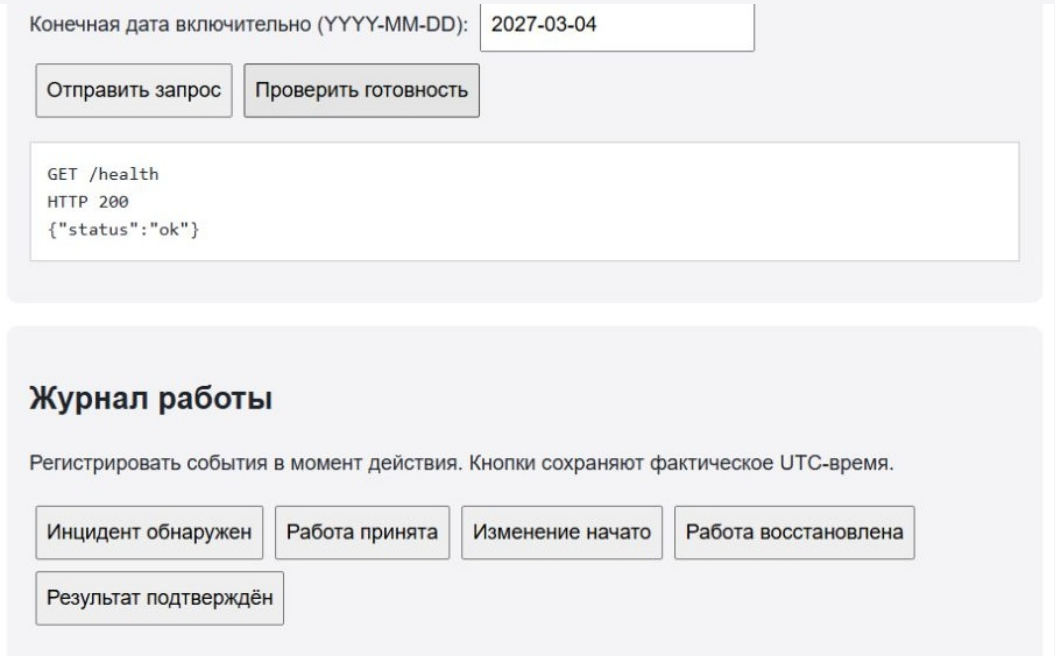



Рисунок 7 — Готовность возвращается после восстановления из копии.

3

## 3.19. Регламент и мониторинг

В отчёте задать объект копии (данные, конфигурация, исходный код, коллекция), момент создания (до изменения), хранение отдельно от рабочей папки и ответственного. Для реальной организации дополнить период, место вне устройства и срок хранения с обоснованием. RPO — допустимая потеря данных по времени, RTO — допустимое время восстановления; пределы объявляются до испытания. В модели копия снята непосредственно до теста, потерю записей проверить по ID/хешу. Мониторинг: /health=503 — немедленная тревога администратору; 500 запросов — разбор данных; 400 — разбор ввода. Дефект состава при 200 обнаруживает граничная коллекция, а не один /health.

# 3. Методика — собственный перенос

1

## 3.20. Изменить условие

Сохранить исходный data.json. В рабочем файле только для третьей записи заменить дату следующего дня на end своей карточки. ID и title оставить. До запроса изменить ожидаемые ids граничного случая в cases.json на все три ID. Выполнить пульт и коллекцию. В примере после переноса ID 103 на 2027-03-04 ожидаются [101,102,103]. Проверка «после всех дат» остаётся с тремя ID. Объяснить, почему результат изменился из-за данных, а не новой правки кода.

2

## Изменённые данные дают три записи на включительной границе.

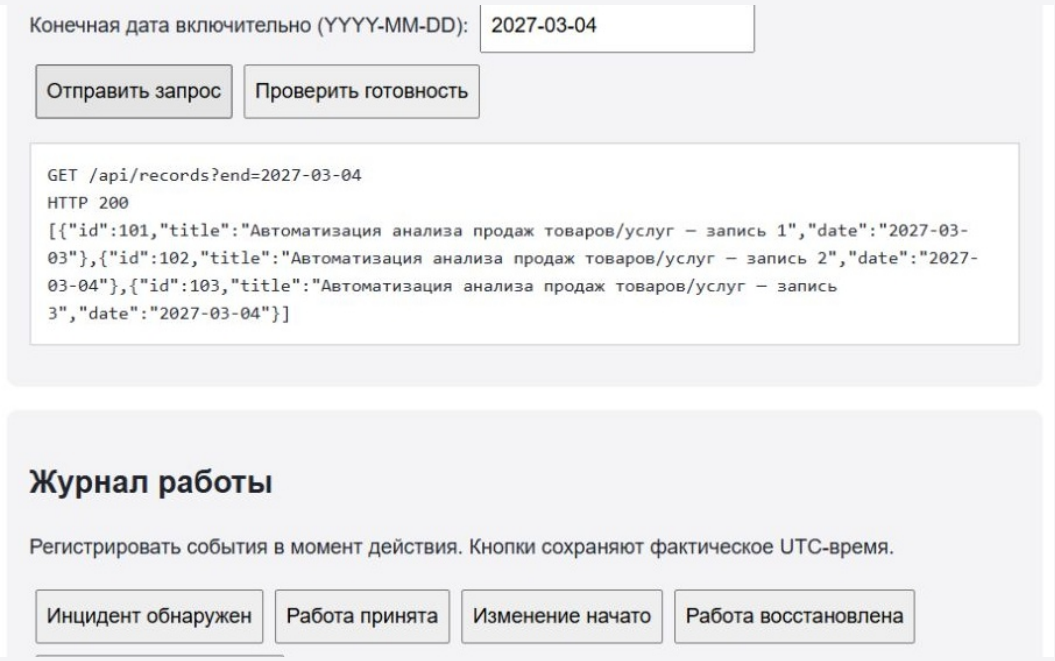



Рисунок 8 — Изменённые данные дают три записи на включительной границе.

3

### 3.21. Вернуть данные и выполнить чистый повтор

Восстановить исходный data.json из backup, вернуть ожидание двух ID в cases.json. Остановить свой сервер. Скопировать исходный проект и исправленный Program.cs, data.json, contract.md, context.json, wwwroot и cases.json в новую папку clean; bin, obj и старый events.jsonl не переносить. В новой папке выполнить dotnet run, открыть пульт и повторить общий запрос, точную границу, ошибочную дату и коллекцию. Проверить отказ/восстановление ещё раз. Сохранить новый журнал отдельно, не заменяя первый.

ПП ПМ.06 · Методические рекомендации

# 3. Методика — фактические показатели

1

### 3.22. Прочитать журнал

Открыть «Журнал событий» в пульте. Каждая строка JSONL — отдельное событие с at в UTC. Сохранить events.jsonl. Для INC-01 взять detected, acknowledged, restored, confirmed. Реакция = acknowledged − detected, восстановление = restored − detected; перевести секунды в минуты делением на 60. Подтверждение проверяющего не включать в момент технического восстановления. Пример расчёта на условных отметках: 10:00, 10:02, 10:12 дают 2 и 12 минут; это объяснение формулы, не результат испытания.

2

### 3.23. Рассчитать и интерпретировать

Сопоставить полученные минуты с пределами своего context.json. Доля успешных проверок = PASS / число выполненных случаев × 100%; до исправления пример 4/5=80%, после 5/5=100%. Для доступности заранее определить окно наблюдения T и подтверждённые интервалы отказа D; (T−D)/T×100%. Для ручных опросов указать шаг опроса и погрешность границы; если начала/окончания отказа нет, показатель не вычислять, написать «данных недостаточно». Не называть один инцидент MTTR за месяц. По фактам предложить минимум две меры: граничный тест перед выпуском и проверенное внешнее резервирование; назвать ответственного и способ проверки эффекта.

3

### 3.24. Закрыть три ситуации

INC-01: пропуск записи, подтверждён предикат, исправление и контрпример. INC-02: файл недоступен, 500/503, восстановление копии, хеши и данные совпали. REQ-03: неверная дата, ожидаемый 400, инструкция правильного ввода; это сервисная помощь, а не дефект API. Для каждой записать вход, ожидаемый и фактический результат, действие, время, ответственного и состояние закрытия. Нарисовать жизненный цикл «Зарегистрирован → В работе → Проверка → Закрыт», с ветвью повторного открытия при неуспехе.

# 3. Методика — схемы и сдача

1

## 3.25. Построить схемы без скрытых шагов

Для каждой схемы в PowerPoint: Главная → Создать слайд → Пустой; Вставка → Фигуры → прямоугольник, ввести подпись; повторить для этапов; Вставка → Фигуры → соединительная стрелка, вести от границы первого блока ко второму. Для развилки взять ромб и подписать стрелки «Да»/«Нет» обычными текстовыми полями. Сохранить PPTX, выделить всю схему Ctrl+A, скопировать в область рисунка отчёта как изображение, не растягивая пропорции. Подготовить контекст, роли, процессы, жизненный цикл, изменение и восстановление. Для последнего: «Отказ → Выбор проверенной копии → Восстановление → Проверка данных», ветвь «Не совпало → другая копия/эскалация», «Совпало → запуск/приёмка». Схемы пометить как блок-схемы; не приписывать им неподдержанную нотацию.

2

## 3.26. Заполнить общий отчёт

В Word заменить подчёркивания и подсказки в полях своими данными. В каждом этапе записать ожидание, действие, факт и объяснение; таблицы расширяются при вводе. Для рисунка выделить текст «Вставить собственный рисунок» внутри рамки → Вставка → Рисунки → Это устройство → свой PNG/JPEG. Подпись под рамкой сохранить; размер выбирать в пределах полей. Ctrl+A → F9 обновляет нумерацию. Проверить отсутствие страницы «Содержание», наличие трёх главных разделов, чёрный шрифт и ссылку репозитория в разделе сдачи. Сохранить как Отчет о выполнении.docx.

3

### 3.27. Проверить передачу

В README указать среду, запуск, контрольную дату, тесты, копию и остановку своего сервера Ctrl+C. Разместить исходники и синтетические данные в своём репозитории; не включать реальные персональные сведения и секреты. Открыть ссылку репозитория в отдельном окне и проверить доступ. Официальные формы из ZIP заполнить своими реквизитами; согласовать реальные подписи с руководителем. Презентация: исходное состояние, причина, правка, тесты, восстановление и показатели. В PowerPoint Показ слайдов → С начала, пройти каждый слайд; Файл → Экспорт → PDF. Передать все файлы из задания.